

基于 ESP32 嵌入式 Web 服务器的手机化仪表设计

丁 依,何岭松

(华中科技大学 机械科学与工程学院,湖北 武汉 430074)

摘 要: 随着物联网技术的发展,单片机性能升级且功能变得丰富,利用单片机创建 Web 服务器,使用浏览器作为客户端进行访问变得可行。借鉴其思路,提出一种嵌入式 Web 服务器+浏览器架构的无软件化手机仪表设计方法。先用 JavaScript 语言将常用的手机仪表元素设计为一个能嵌入到单片机存储系统中的 50 KB 大小的库,然后在其基础上形成 C 风格手机仪表 HTML 网页生成函数,最后再通过单片机 Web 服务器将封装后的 C 风格手机仪表 HTML 网页生成函数转换为手机浏览器支持的 HTML 网页进行显示和用户操作。该设计实现将仪表软件安装在下位机,客户端零安装、零配置访问仪表界面。

关键词: 嵌入式; Web 服务器; 手机仪表; 浏览器; JavaScript 程序; 仪表库

中图分类号: TP311.1

文献标志码: A

DOI: 10.16157/j.issn.0258-7998.256290

中文引用格式: 丁依,何岭松. 基于 ESP32 嵌入式 Web 服务器的手机化仪表设计[J]. 电子技术应用, 2025, 51(11): 107-115.

英文引用格式: Ding Yi, He Lingsong. Design of mobile phone instrument based on ESP32 embedded Web server[J]. Application of Electronic Technique, 2025, 51(11): 107-115.

Design of mobile phone instrument based on ESP32 embedded Web server

Ding Yi, He Lingsong

(School of Mechanical Science and Engineering, Huazhong University of Science and Technology, Wuhan 430074, China)

Abstract: With the development of Internet of Things technology, the performance of the single-chip microcontroller has been upgraded and the functions have become rich, and it has become feasible to use the single-chip microcontroller to create a web server and use the browser as a client to access. Drawing on its ideas, a mobile phone instrument design method that does not require software installation based on embedded Web server + browser architecture was proposed. First, the JavaScript language is used to design the commonly used mobile phone instrument elements as a 50KB library that can be embedded in the storage system of single-chip microcontroller. Then, on the basis of it, the C-style mobile phone instrument HTML web page generation function is formed. Finally, through the single-chip microcomputer web server, the encapsulated C-style mobile phone instrument HTML web page generation function is converted into the HTML web page supported by the mobile phone browser for display and user operation. It realizes the function of installing the instrument software on the lower computer, and accessing the instrument interface with zero installation and zero configuration on the client.

Key words: embedded; web server; mobile phone instrument; browser; JavaScript program; instrument library

0 引言

随着社会需求不断地信息化、自动化和智能化,基于单片机设计的智能仪表在不同领域中都发挥着重要作用^[1-3]。目前主流的智能仪表大多需要在显示模块上预装 APP 后才能使用^[4-5],对安装平台也有一定的要求^[6],当下位机仪器被更换时,需要重装软件进行适配,较为繁琐,或者存在整个仪表的集成度太高导致二次开发的学习成本较大等问题,例如东辉的 DY2000 系列智能仪表,通常使用 LCD、LED 或 OLED 显示屏作为显示模块,界面设计简洁而直观,但显示的内容有限且更改较为困难,用

户在某些应用场景下可能无法找到合适的功能设置;德能的智能水表采用蓝牙技术与手机连接的设计模式,仪表界面更加丰富,实现了“抄表不入户,缴费不出门”的智能供水服务,但需要下载并安装 APP 后才能使用。随着物联网技术的迅速发展,目前许多单片机都集成了 ESP32、ESP8266 等 Wi-Fi 通信模块,且 ESP32 的大部分性能优于 ESP8266^[7-8],能够轻松地在下位机运行嵌入式 Web 服务器,实现网页托管与访问等功能,网上已经有了许多应用^[9-11]。由于嵌入式系统资源有限且仪表库封装工作繁琐,目前没有能够在嵌入式 Web 服务器中运行的

小型仪表库,缺少这类手机仪表的应用。

针对这些问题,本文提出一种基于ESP32嵌入式Web服务器的手机化仪表设计方法,采用嵌入式Web服务器与浏览器通信的模式,设计了一个可以自由组装仪表界面的下位机标准库,能够将仪表界面存储在单片机中,通过Web服务器方式运行,利用手机浏览器查看与操作仪表界面,不用安装任何软件,只需要连接与下位机相同的热点,即可通过设备IP地址访问不同仪器的仪表界面。该设计操作简单且交互友好,学习成本低,仪表库开源便于进一步拓展。

1 嵌入式Web服务器手机仪表总体架构

本手机仪表基于普通嵌入式Web应用的技术架构进行设计,在单片机上创建并运行Web服务器,用户通过浏览器访问嵌入式Web服务器获取网页。嵌入式Web服务器手机仪表的总体架构如图1所示,为了便捷地实现一个仪表界面,本设计利用HTML元素+JS(JavaScript)程序模块化封装了一个小型Web仪表库,由于下位机开发环境只能运行下位机程序,因此利用开源库函数或者其他工具将JS仪表库引入下位机项目,再将上位机仪表库封装成下位机函数库,从而实现能够下位机IDE中快速组装出一个仪表界面。采用HTTP协议响应客户端仪表界面和处理简单请求,当用户访问页面时,Web服务器读取界面程序,并将程序转换成HTML文件返回给用户。所有的仪表库和程序都存储在单片机上,从而实现了客户端的无软件化。再利用WebSocket协议作为客户端与服务器之间实时传输数据与指令的通信协议,用户点击按钮发送WebSocket指令,嵌入式设备根据指令采集传感器数据或控制设备,并通过WebSocket将数据实时显示在页面上。

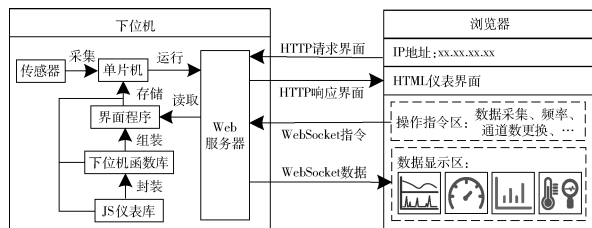


图1 手机仪表总体架构图

图2为手机仪表的使用流程图。将手机设备连接到下位机已连接的Wi-Fi网络上,使上、下位机处于同一局域网,打开浏览器,输入下位机设备的IP地址,此时客户端向下位机发送HTTP请求,下位机监听到请求并将页面文件进行响应,当客户端完整显示仪表界面后,就可以开始进行仪表操作。操作前先建立与下位机的WebSocket通信连接,实现实时的双向通信,然后开始根据需求发送指令、接收数据、处理数据,并在仪表上可视化。单片机与不同类型的传感器相连构成不同的下位机仪器,利用设计的仪表库组装各自的仪表界面,访问对应仪器的IP地址就可以查看界面并操作,不用安装多个仪表APP。

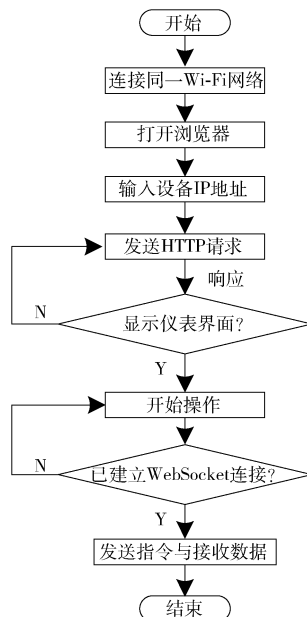


图2 仪表的使用流程图

2 手机仪表无软件化应用实现

2.1 微型JS仪表库设计

目前Web端的仪表界面大多是利用JS开源可视化库搭建的^[12-13],这类开源库资源占用较多,图3是利用ECharts库组装的虚拟调试界面^[14],需要仪表盘、输入控件、波形图、数值显示框等仪表控件。在嵌入式Web服务器应用中,由于内存问题不能直接引用这些开源库,因此只能自己实现基本的仪表库。目前主要用于物理量测量领域,例如心电图表、压力计、加速度表、数字示波器等,所以该仪表库具备了常见的波形图、仪表盘、滑块、按钮等仪表控件。整个仪表库的大小在50 KB左右,适用于大部分测量领域和场景。



图3 ECharts仪表界面

下面以温度计为例对JS仪表库的封装逻辑进行介绍,使用HTML的<canvas>元素作为仪表控件的容器,<canvas>元素定义了一个空白绘图区域,可以通过JS程序在这块区域内进行图形绘制和动画显示。先对<canvas>在绘制仪表的过程中常见的画图动作进行封装,如图4所示的文本绘制、圆形绘制、直线绘制等动作,方便快速引用以及减少代码量,把ctx对象、字体样式、对齐方式、绘制内容、绘制位置等配置参数设为形参,将一些使用时相同值较多的参数设置成默认值,方便调用。

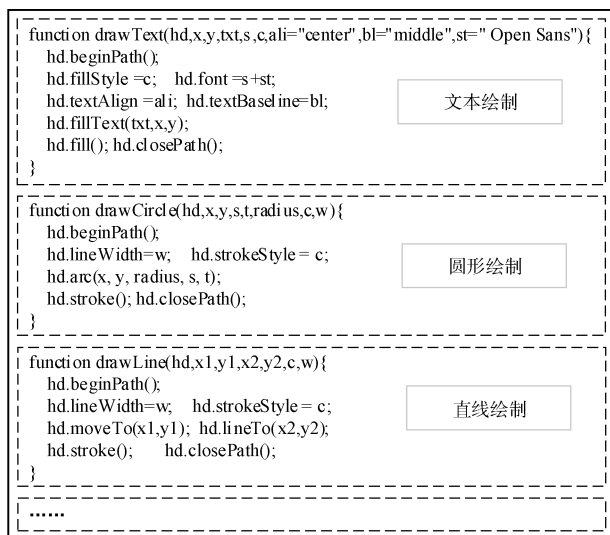


图4 canvas常见绘制动作封装

利用上述的绘制函数,用较少的代码就实现了温度计仪表函数的组装,其组装程序如图5所示。

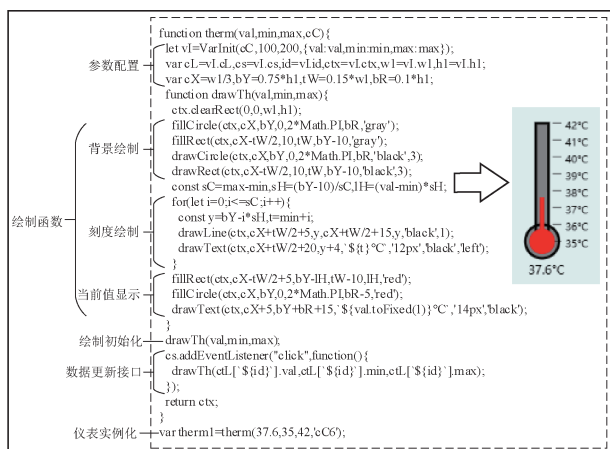


图5 温度计组装程序与实例图

首先进行参数配置,创建并初始化仪表 canvas 元素,赋予其宽度、高度、id 值,获取 ctx 对象,初始化其数据存储对象,并定义好绘制过程中用到的变量;然后利用这些参数封装仪表的绘制函数,包括背景绘制、刻度绘制、当前值显示,封装完后调用它进行初始化,在之后的刷新仪表事件中,也是通过调用此函数进行更新;最后利用 DOM 的 click 事件作为数据更新后立即回调绘制函数的事件,成为数据更新的外部接口。留出当前值、最大值以及父容器作为整个仪表函数的形参,最后通过一行代码就能完成仪表的实例化。把 ctx 对象作为函数的返回值,用于后续数据刷新函数的输入值,温度计的刷新函数如图6所示,通过 ctx 对象获取已实例仪表的 id 值和 canvas 元素, id 值用于数据子对象的区分, ctL ['\$id'] 就是对应 id 仪表的数据对象,更新该对象内的数据,并通过 canvas 触发 click 事件实现仪表更新。

根据上述设计思路,对其余仪表进行封装,就完成

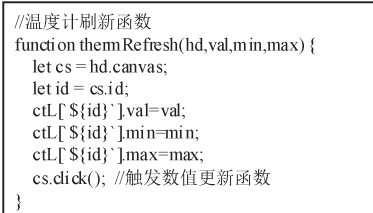


图6 仪表刷新函数

了JS仪表库的整体封装,其显示效果如图7所示。

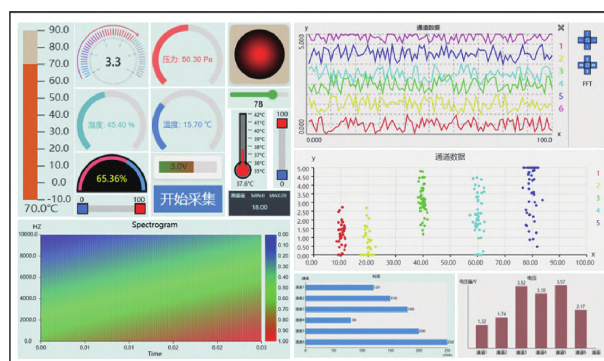


图7 JS仪表库部分展示

2.2 下位机标准库封装

2.2.1 JS 仪表库的引入

由于大多数单片机开发环境无法直接运行 JS 程序,需要依赖下位机程序运行,因此需要将 JS 仪表库转换成下位机函数。首先将 JS 仪表库引入下位机项目,目前有许多开源库和工具支持这一操作,例如 EmbeddedFiles、incbin 等函数库, LittleFS、SPIFFS、FlashStorage 等文件系统库,以及将 JS 文件转换为头文件的工具。所选用的方式取决于具体使用的硬件平台、性能需求和项目目标。现在以 incbin 库为例对整个引入流程进行介绍,如图8所示,将 JS 脚本以 txt 文本形式保存,通过文件名称和存储相对位置获取文件内容,并以二进制形式嵌入程序,获取指向仪表库内存区域的指针,在下位机函数的封装过程中,利用这些指针在系统中读取 JS 仪表库资源。

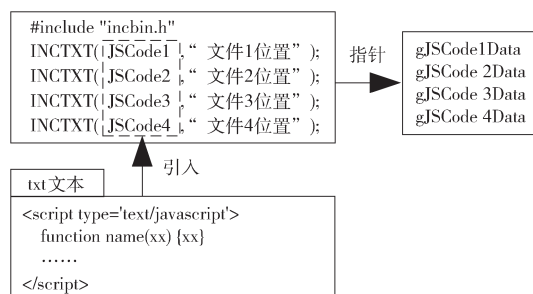


图8 JS 仪表库引入流程

2.2.2 下位机函数封装原理

引入 JS 仪表库后,在其基础上形成 C 风格的手机仪表 HTML 网页生成函数,将仪表中常用的 HTML 元素和 JS 程序包装上传给客户端的动作外衣,直接作为标准

库函数的函数体,函数的形参接口包含HTML元素与JS函数留出的所有形参,便于网页的实例化。JS仪表库函数的实例引用一般为一句代码,节约存储空间。当调用

这些下位机函数组装网页时,实际调用的是里面对应的JS程序与HTML元素。按照这种封装思路,形成的下位机函数及其使用功能如表1所示。

表1 下位机函数库

下位机库函数	对应的HTML元素+JS程序
insertJSLib()	JS仪表库嵌入HTML文档函数
httpResponse()	向客户端发送HTTP响应头程序
htmlStart()	HTML文件开始设置,包括文档类型声明、头部区域设置、主体区域开始
htmlStyle()	HTML元素样式设置,将所有的class都在此处声明
htmlEnd()	HTML文件结束设置,包括关闭主体区域、HTML标签区域
uiContainer()	仪表界面固定元素设置,包括布局容器、仪表名显示标签、WebSocket连接按钮、连接状态显示标签等元素
varSet()	全局变量声明函数,在此处进行所有全局变量的设置,包括仪表数据对象初始化、WebSocket对象变量名定义、通过id获取HTML元素、仪表参数范围设置等
ws_onmessage()	客户端接收并处理消息函数,在这个函数里根据仪表需求自定义函数体,包括消息接收判断逻辑、数据处理逻辑,并将处理好的数据绑定相应的仪表刷新函数,实现下位机数据与页面的联动
button()	按钮控件实例化函数
dashboard()	仪表盘实例化函数
valTab()	数值显示框实例化函数
digitalTab()	数字显示框实例化函数
LED()	警示灯实例化函数
thermometer()	温度计实例化函数
textBox()	文本显示框实例化函数
scrollWin()	数据滚动框实例化函数
waveform()	波形图实例化函数
scatter()	散点图实例化函数
colChart()	柱状图实例化函数
barChart()	条形图实例化函数
barDiagram()	棒图实例化函数
bitColorMap()	位彩色图实例化函数
sliderDoubleV()	双头竖直滑块实例化函数
sliderDouble()	双头滑块实例化函数
Slider()	滑块实例化函数
startTime()	当前时间更新函数
jsStart()	JS脚本程序开始标识
jsEnd()	JS脚本程序结束标识
htmlReturn()	换行标签
htmlLine()	水平分隔线标签
linkLink()	超链接标签
htmlHeadLine()	标题标签
htmlText()	段落标签
htmlForm()	表单标签
formSubmit()	表单提交控件
htmlLabel()	标签元素,通常与输入控件相关联,用于功能提醒
textInput()	单行文本输入框
numberInput()	数字输入框,一般作为向下位机传输消息的控件,例如采样频率数值输入框
htmlCanvas()	绘制图形标签
htmlDiv()	块级容器标签
htmlCheckbox()	复选框控件,常用于选择通道数
htmlSelect()	单选下拉列表控件,常用于切换参数与控件类型
boxStart()	块级容器开始标识
boxEnd()	块级容器结束标识
addEvent()	添加事件监听器函数
addEventStart()	自定义事件函数开始标识
addEventEnd()	自定义事件函数结束标识

整个函数库包括组成网页界面的各部分元素,例如网页文档声明、网页布局元素、网络通信接口、实例化仪表、元素事件绑定接口等。其中 `ws_onmessage()` 函数是整个仪表页面的核心,客户端收到下位机数据消息后根据类型进行处理并分别传入对应仪表刷新函数,是采集数据可视化的关键。标准库中的函数大部分可重载,按需求更换形参来增删样式与功能。使用者通过下位机程序简单、快速地组装出网页仪表界面,不用过多关注上位机语言。以简易温度计界面为例进行说明,其下位机组装程序与对应的 HTML+JS 程序如图 9 所示,组成 `sendWebPage()` 函数的所有资源都存储在单片机上,手机不用安装任何应用。

页面组成函数首先对 HTML 文档进行声明,定义基础的 HTML 页面结构;引入页面 class 样式,在 HTML 元

素创建与 JS 仪表库封装中都有调用;设置仪表固定的 HTML 布局元素,例如仪表容器、仪表标题显示标签、WebSocket 连接按钮与状态显示标签,点击连接按钮,创建对应 IP 地址的 WebSocket 实例并连接,实现与下位机的实时通信,还可以根据需求使用函数库增加 HTML 元素,例如采样频率输入框等;引入 JS 仪表库,里面包含仪表所需的固定事件函数、通信函数和所有用于实例化的仪表函数;开始 JS 程序,先定义和初始化全局变量,例如仪表的数据对象 `ctL`、WebSocket 实例对象 `ws`、仪表 id 值 `index1`、温度计的最值范围 `yMin` 和 `yMax`;消息处理函数 `ws_onmessage()` 对收到的消息进行判断,进入对应的数据处理逻辑分支,本例中若为单值数据消息,则转换为数值后与温度计刷新函数绑定,实现温度数值的显示与刷新;最后实例化开始采集按

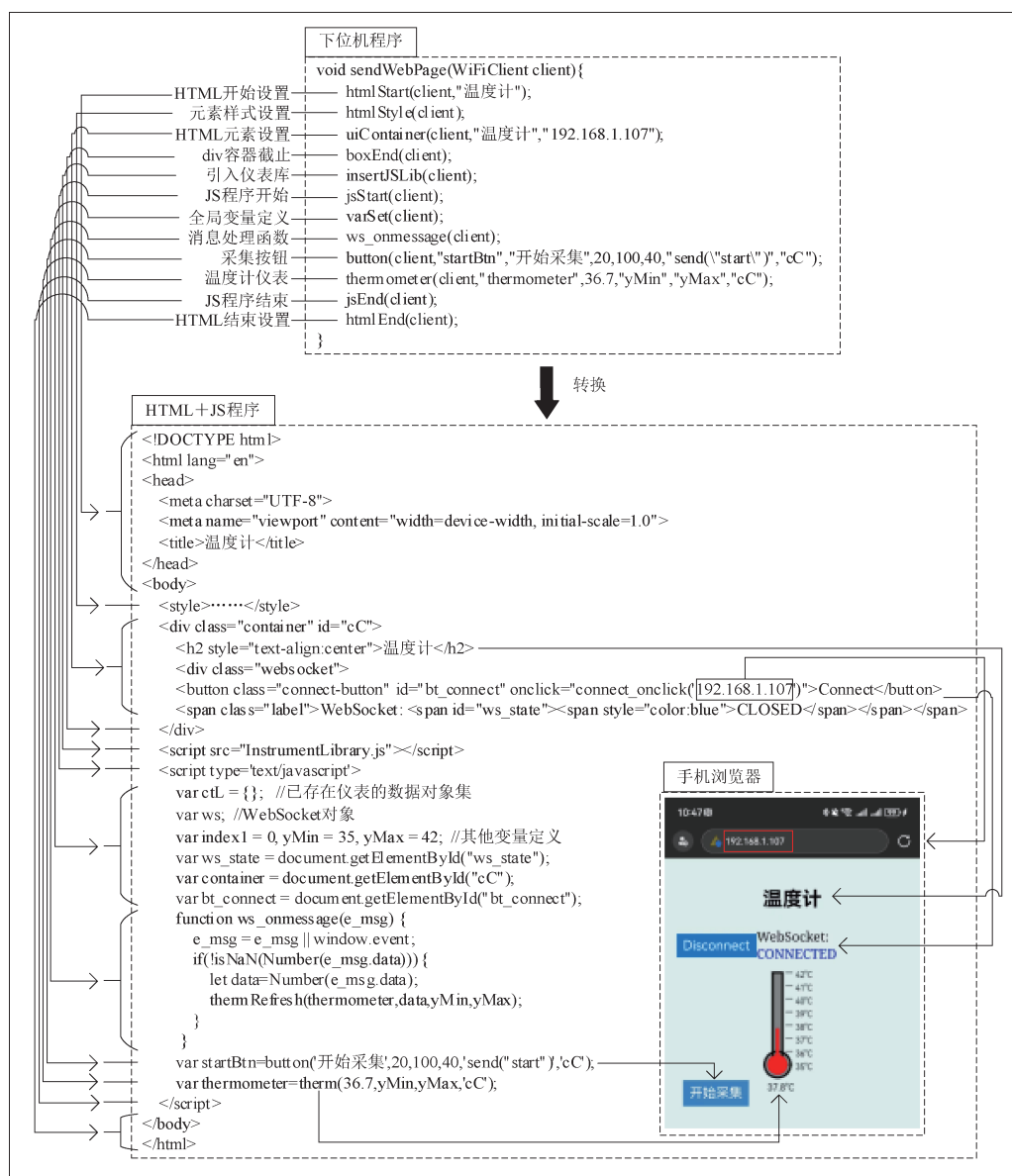
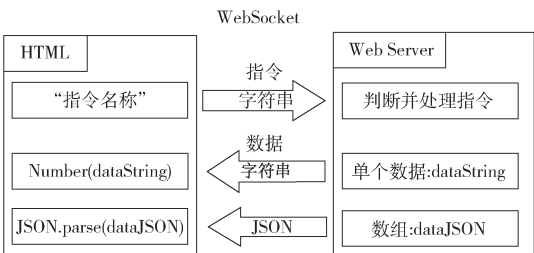


图9 温度计组装函数对应图

钮和温度计仪表。由此完成了客户端的整个仪表界面设计与功能设置。

2.2.3 上下位机的数据传输格式

上下位机间数据传输格式如图 10 所示,客户端需要实时更新数据时,将单个数据以字符串格式传输,上位机通过 `!isNaN(Number(data))` 条件进行判断,并利用 `Number(data)` 函数将其转换成数字。当实时性需求不高时,将采集数据存储在一定长度的数组里并以 JSON 格式传输,上位机通过 `data.indexOf(",")!=-1` 条件进行判断,并利用 `JSON.parse(data)` 函数将其解析成数组。



上位机通过协议接口 `ws.send()` 函数将指令以字符串格式发送给下位机,下位机通过硬件接口与传感器或执行器相连,进行数据采集或者设备控制,然后转发数据给上位机。下位机对收到的指令信息进行判断,对于尾端没有数字的指令,例如“start”“stop”等,判断是否存在相同设定并作出对应操作;对于尾端含有数字的指令,例如“S100”“N1”等,判断字符串是否包含既定字母或组合,提取出字母后面的数字作为参数值,再根据字母对应指令含义进行操作。仪表页面中常见的指令集如表 2 所示,主要实现数据采集启停、通道数选择及采样频率设置。

2.3 Web 服务器转换网页实现原理

以温度计仪表为例对仪表在 Web 服务器里的运行原理进行说明,首先搭建对仪表应用的创建环境,如图 11 所示,在下位机项目中引入库资源,其中 `<WiFi.h>` 用于配置开发板连接 Wi-Fi 网络,创建热点或连接已有的

网络;`<WebSocketServer.h>`用于在开发板上实现 Web-Socket 服务器;“`htmlLib.h`”是设计的下位机标准库,用于仪表网页构建。在全局变量定义区指定设备连接的 Wi-Fi 网络名称与密码;在端口 80 上创建用于处理 HTTP 请求的服务器实例,向客户端提供网页;在端口 81 上创建用于处理 WebSocket 连接的服务器实例,实现客户端和服务端之间实时双向通信的持久连接,向客户端网页提供数据;定义下位机的采样频率、采集通道、采样状态标识初始值。

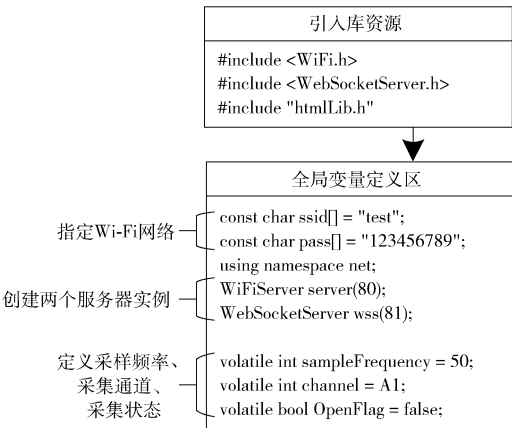


图 11 仪表创建环境初始化

对两个服务器进行初始化配置与启动,如图 12 所示,先连接到 Wi-Fi 网络,再启动 HTTP 服务器;配置 WebSocket 连接、接收消息和关闭事件,例如本例中对开始采集指令消息进行处理,利用布尔值变量 `OpenFlag` 作为数据采集状态标识,服务器收到“start”指令将 `OpenFlag` 变为 `true`,开始采集状态;所有配置完成后启动 WebSocket 服务器。

最后对两个服务器的具体使用功能进行设计,如图 13 所示,在循环执行程序中不断监听 WebSocket 客户端请求;检查是否有新的 HTTP 客户端连接,如果有客户端连接到 HTTP 服务器,读取其请求头并发送 HTTP 响应头和仪表网页, `sendWebPage(client)` 函数在服务器里运行时是 C 风格下位机程序,发送给手机客户端的是对应

表 2 WebSocket 通信指令集

指令	备注
start	开启数据采集
stop	停止数据采集
N...	选择通道
N123	
N456	
S...	设置数据采样频率
S100	
S2000	

1~6 分别代表 1~6 号通道;N123 表示选择 1 号、2 号和 3 号通道;N456 表示选择 4 号、5 号和 6 号通道

定的情况下,通道数越多,则最高频率越低

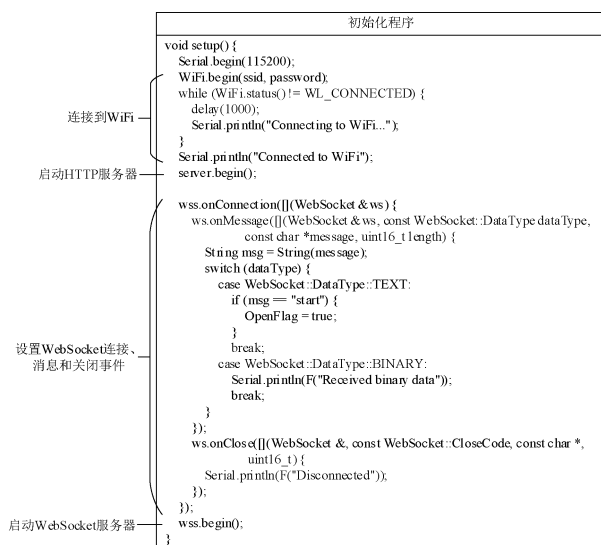


图 12 服务器配置与启动程序示例

提取出来的 JS 程序和网页元素,是一个完整的 HTML 文件内容,其在浏览器中运行并渲染成仪表 GUI 界面;使用 millis() 函数实现采样间隔控制,服务器按照采样频率读取 A1 通道的采集数据,通过 WebSocket 广播将数据传输给所有连接的客户端。由此就实现了整个仪表的创建与运行。

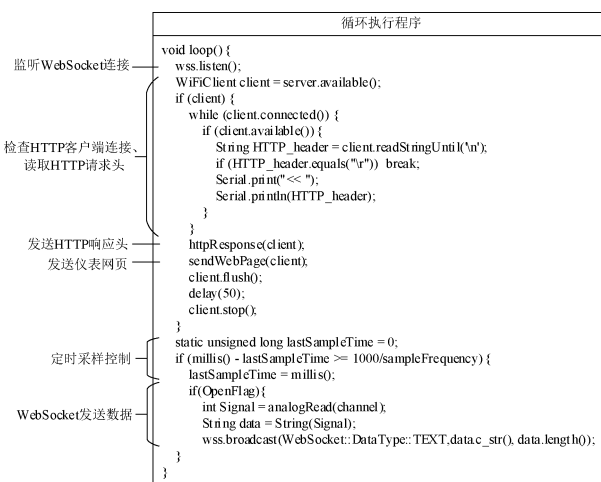


图 13 服务器功能使用程序示例

3 应用实例与验证

3.1 心跳脉搏仪应用实例

为了验证此手机仪表设计的可行性、使用的便捷性及仪表库的适用性,基于上面的设计方法,以 Arduino UNO R4 WiFi 板卡作为嵌入式下位机为例进行应用介绍。Arduino UNO R4 WiFi 上的处理器是 RA4M1 微控制器,具有 48 MHz 的时钟速度、32 KB 的 SRAM 和 256 KB 的闪存;集成的 ESP32-S3 模块是 ESP32 的变体,在性能和功耗方面进行了改进,适合较复杂的项目,且 Arduino 生态系统中有许多专用的库支持 Web 服务器功

能。整体价格在 190 元左右,是性价比很高的一款开发板,使用它来做手机仪表的下位机、创建 Web 服务器非常方便。依据上述设计原理实现了心跳脉搏仪应用,整个应用所需的设备如图 14 所示,包括一个脉搏传感器、一块 Arduino UNO R4 WiFi 开发板和一台手机。



图 14 仪表应用所需硬件

使用下位机标准函数库组装其仪表界面,并在 Web 服务器中运行,手机浏览器访问设备的 IP 地址,接收到 Web 服务器传来的 HTML 内容并生成仪表网页,在手机上的显示效果如图 15 所示。



图 15 心跳脉搏仪界面

WebSocket 已连接,设置采样频率为 50 Hz,点击按钮开始采集数据,下位机程序利用 <PulseSensorPlayground.h> 库处理从脉搏传感器获取的原始数据,提供计算心率、检测脉搏信号的功能,将处理好的数据实时传输给浏览器。心跳数据在波形图上显示,测量响应速度快且显示直观,最下方的滚动框实时更新心率数据。图 15 中,左图当前心率为 73 bpm 时,处于正常范围,显示心率正常,警示灯熄灭;右图当前心率为 48 bpm,低于正常范围,显示心率异常,警示灯亮起。整个仪表应用的使用效果较为理想,数据交换过程简洁流畅,有较好的人机交互体验。

3.2 其他测量领域应用实例

在很多小型测量领域,例如微小压力、电流、电压、光强、温度、湿度等测量,都可以根据需求选择

合适的仪表控件组装仪表界面,可以导出数据用于较为复杂的分析。单片机连接不同类型的传感器实现不同的仪表应用,如图16所示的压力表、温度计、多功能测量仪表,可以测量单一的量,也可以多通道采集同时测量多种量,仪表的组装与操作都很方便。



图16 手机仪表应用展示

4 结论

本文提出的基于ESP32嵌入式Web服务器的手机仪表设计,对传统的智能仪表进行了改进。利用原生HTML和JS程序封装小型仪表库,并将其转换成下位机标准库存储到单片机上,相当于提前将仪表APP安装在了下位机,不用在显示设备上进行任何安装,实现了零安装零配置。利用ESP32模块的无线通信功能,在下位机上运行一个轻量级的Web服务器,用户通过浏览器访问设备的IP地址就可以获取仪表控制界面,对物理量进行测量、显示与简单控制设备。上位机界面程序模块化开发,仪表界面可以通过下位机函数库自由组装,对应不同的应用场景。此设计方法通过使用Arduino UNO R4 WiFi板卡进行实例展示,在线下的实验课中进行了教学应用,具有较好的使用效果。上下位机之间通过WiFi连接,支持多种标准协议通信,未来便于对仪表功能进行拓展^[15-16]。可以利用云计算^[17],实现实时数据的复杂处理分析与预测,以及可以增加MQTT协议^[18],实现与工业系统或者智能家居生态互联互通。通过进一步地集成智能化、互联化和模块化设计,这种仪表系统将能够适应更多复杂的应用场景,具有很大的应用前景。

参考文献

- [1] 张应华. 基于电子技术的嵌入式单片机应用分析[J]. 电子元器件与信息技术, 2021, 5(5): 107-108.
- [2] 何璐兵. 单片机在智能仪表方面的应用分析[J]. 中国设备工程, 2021(7): 38-39.
- [3] SONG Z J, XU L J, YAN L G. Design of a simple and multifunctional liquid parameter measuring instrument based on MSP430 single chip microcomputer[C]//2021

International Conference of Social Computing and Digital Economy (ICSCDE). IEEE, 2021: 121-124.

- [4] 李腾腾, 蔡梦杰, 高永峰, 等. 基于智能手机视频处理技术的智能电表故障检测[J]. 电气技术, 2021, 22(6): 124-127.
- [5] PAUL A, TIWARI R. Smart home automation system based on IoT using chip microcontroller[C]//2022 9th International Conference on Computing for Sustainable Global Development (INDIACom). IEEE, 2022: 564-568.
- [6] 贾世盛. 工业自动化仪表与自动化控制技术研究[J]. 机械与电子控制工程, 2024, 6(24): 34-36.
- [7] 杨子弘, 胡诚羽, 刘佳. 基于ESP32的虚拟电子围栏设计与实现[J]. 计算机科学与应用, 2021, 11(7): 1962-1972.
- [8] NIRANJANA R, ARVIND S, VIGNESH M, et al. Effective home automation using ESP32 NodeMCU[C]//2022 International Conference on Automation, Computing and Renewable Systems (ICACRS). IEEE, 2022: 1-5.
- [9] 张前程, 周琬玥, 黄杰民, 等. 嵌入式颈椎健康智能系统设计[J]. 医疗卫生装备, 2024, 45(8): 26-31.
- [10] LI G L, ZHOU C, MOU F Y. Design of remote monitoring and control system for agricultural greenhouse environment based on ESP32[J]. Journal of Chinese Agricultural Mechanization, 2022, 43(3): 47.
- [11] AKBAR A, ZAENUDIN Z, MUTAQIN Z, et al. IoT-based smart room using web server-based ESP32 microcontroller[J]. Formosa Journal of Computer and Information Science, 2022, 1(2): 91-98.
- [12] XIE J, YANG Z, JUN Z, et al. Design and implementation of epidemic big data visualization platform based on Vue+ ECharts[C]//2023 IEEE 2nd International Conference on Electrical Engineering, Big Data and Algorithms (EEBDA). IEEE, 2023: 1347-1352.
- [13] ZHANG J, SHENG Y, CHEN W, et al. Design and analysis of a water quality monitoring data service platform[J]. Comput. Mater. Continua, 2021, 66(1): 389-405.
- [14] 孙雪皓, 张凤丽, 周正飞, 等. 数控机床数字孪生建模技术及智能应用研究[J]. 计算机集成制造系统, 2024, 30(3): 825-836.
- [15] 刘浩磊. 5G技术在电力系统中的应用[J]. 机械与电子控制工程, 2022, 4(5): 169-171.
- [16] 李春艳. 关于自动化仪表控制系统技术研究[J]. 机械与电子控制工程, 2024, 6(18): 209-211.
- [17] HASHMI S A, ALI C F, ZAFAR S. Internet of Things and cloud computing-based energy management system for demand side management in smart grid[J]. International Journal of Energy Research, 2021, 45(1): 1007-1022.

- [18] GAIKWAD K. IoT based water management system using MQTT protocol[C]//2021 5th International Conference on Trends in Electronics and Informatics (ICOEI). IEEE, 2021: 408-414.

(收稿日期:2025-01-14)

作者简介:

丁依(1999-),女,硕士研究生,主要研究方向:嵌入式开发、信号处理、计算机虚拟仪器技术。

何岭松(1962-),通信作者,男,教授,主要研究方向:工业测量与控制、虚拟现实技术、设备故障远程诊断, E-mail: 2394706156@qq.com。



扫码下载电子文档